

Day 02 Practice Assignments: String Manipulations

Objective

Apply string indexing, slicing, concatenation, repetition, formatting, and built-in string methods to solve problems.

Part A: Easy Complexity (3 Exercises)

Exercise 1: Sentence Analysis (Character & Word Count)

Write a Python program that prompts the user to enter a sentence. The program must count and display:

1. The total number of characters (including spaces and punctuation).
2. The total number of words.

- **Sample Input:** "Learning Python is fun!"
- **Sample Output:**

```
Total Characters: 23
Total Words: 4
```

Exercise 2: Reversed Uppercased String

Write a program that takes a string input from the user, reverses the string, converts the entire reversed string to uppercase, and prints the result.

- **Sample Input:** "Bangalore"
- **Sample Output:** "EROLAGNAB"

Exercise 3: Email Domain Extractor

Write a program that prompts the user to enter an email address string. Extract the domain name (the part after the @) and print it. If the string is not a valid email (does not contain exactly one @), print "Invalid Email".

- **Sample Input:** "vinod@vinod.co"
 - **Sample Output:** "vinod.co"
 - **Sample Input:** "vinod.co"
 - **Sample Output:** "Invalid Email"
-

Part B: Medium Complexity (5 Exercises)

Exercise 4: Vowel & Consonant Frequency

Write a program that prompts the user to enter a string and counts:

1. The individual frequency of each vowel (a, e, i, o, u), case-insensitively.
2. The total count of all consonants.

- **Sample Input:** "Vinod Kumar Kayartaya"
- **Sample Output:**

```
Vowel Frequencies:  
a: 4  
e: 0  
i: 1  
o: 1  
u: 1  
Total Consonants: 12
```

Exercise 5: Custom Title Case Formatter

Write a program that accepts a string input from the user and outputs it in Title Case (capitalizing the first letter of each word and lowercasing the remaining letters). **Do not use Python's built-in `.title()` method.**

- **Sample Input:** "WELCOME TO BANGALORE CITY"
- **Sample Output:** "Welcome To Bangalore City"

Exercise 6: Shift Cipher Encrypter

Write a program that prompts the user for a text string and a shift integer, and encrypts the text using a Caesar cipher. It should shift each alphabetical character in the string by the specified shift number down the alphabet. Maintain uppercase and lowercase characters, and leave spaces or punctuation marks completely unchanged.

- **Sample Input:** (User inputs string "Vinod" and shift 3)
- **Sample Output:** "Ylqrg"

Exercise 7: Manual Substring Counter

Write a program that prompts the user to enter a main text string and a substring. Count how many times the substring appears in the main string **without using Python's built-in `.count()` method.**

- **Sample Input:** (User inputs main string "banana" and substring "an")
- **Sample Output:** 2

Exercise 8: Name Anonymizer

Write a program that prompts the user to enter a full name (first name, middle name, last name) and anonymizes it. The output should print the initials of the first and middle names followed by the full last name. If the name consists of only a single word, print it as-is.

- **Sample Input:** "Vinod Kumar Kayartaya"
- **Sample Output:** "V. K. Kayartaya"

- **Sample Input:** "Bangalore"
 - **Sample Output:** "Bangalore"
-

Part C: Difficult Complexity (2 Exercises)

Exercise 9: Longest Palindromic Substring

Write a program that prompts the user to enter a text string and finds the longest substring within it that reads the same forward and backward. If there are multiple palindromic substrings of the same maximum length, print any one of them.

- **Sample Input:** "babad"
- **Sample Output:** "bab" (or "aba")
- **Sample Input:** "cbbd"
- **Sample Output:** "bb"

Exercise 10: Run-Length String Compression

Write a program that prompts the user to enter a text string and compresses it using run-length encoding (listing character counts next to each repeated character). If the compressed string is not smaller in size than the original string, print the original string.

- **Sample Input:** "aabcccccaaa"
 - **Sample Output:** "a2b1c5a3"
 - **Sample Input:** "abcd"
 - **Sample Output:** "abcd" (since "a1b1c1d1" is longer than "abcd")
-

Part D: Challenge Complexity (2 Exercises)

Exercise 11: Group Anagrams

Write a program that starts with a list of strings defined at the top of your script (e.g., `words = ["eat", "tea", "tan", "ate", "nat", "bat"]`) and groups the anagrams (words formed by rearranging letters) together. Print the final grouped list of lists.

- **Hardcoded Input:** `words = ["eat", "tea", "tan", "ate", "nat", "bat"]`
- **Sample Output:** `[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]`

Exercise 12: Date Validator & Pretty Formatter

Write a program that prompts the user to enter a date string in the format "DD/MM/YYYY".

[!WARNING] Do not use any built-in date/time library functions (such as the `datetime` or `time` modules) to format or validate the dates. You must parse and split the string manually, and use a custom tuple of month names for the pretty output if needed.

Your program must:

1. Verify if the date is valid. To be valid:

- The month must be between 1 and 12 inclusive.
 - The day must be valid for that specific month (e.g., April, June, September, November have 30 days; others have 31 days).
 - For February, the day must be at most 29 in a leap year (divisible by 4, except for centuries not divisible by 400) and at most 28 in standard years.
2. If the date is valid, use a tuple of month names ("January", "February", ...) to format and print the date in a long-form readable layout: "MonthName DD, YYYY".
 3. If the date is invalid, print "Invalid Date".
- **Sample Input:** "26/08/2026"
 - **Sample Output:** "August 26, 2026"
 - **Sample Input:** "29/02/2026" (2026 is not a leap year)
 - **Sample Output:** "Invalid Date"
 - **Sample Input:** "31/04/2026" (April only has 30 days)
 - **Sample Output:** "Invalid Date"